NEXORA PLATFORM

v1.0 Production System Specification, Features Guide & IIS Hosting Manual

Status:	Production Ready (v1.0.0 Tagged)
Target Framework:	.NET 10 (ASP.NET Core 10 Web API & Blazor Server)
Mobile Platform:	Flutter 3.47.1 (Dart 3.13.1)
Database:	Microsoft SQL Server 2022 (EF Core 10 Migrations)
IIS Publishing:	A:\Projects\Nexora\publish\api & \admin
Date:	October 2026

Executive Summary: *Nexora is a private, secure, and extensible unified personal and couple platform. It bridges personal productivity (Notes, Reminders, Knowledge Vault), relationship intimacy (Pairing, Shared Calendars, SignalR Chat), curated intelligence (Sector News, Global Search), and AI coaching (Ollama / OpenAI / Graceful Fallback). This document provides an exhaustive breakdown of the 13 engineering phases delivered and comprehensive instructions for local IIS web hosting.*

1. System Architecture & Engineering Principles

Nexora follows strict **Clean Architecture** and **CQRS (Command Query Responsibility Segregation)** patterns using MediatR. The architecture completely decouples domain entities from infrastructure persistence, ensuring modularity, testability, and enterprise maintainability.

Layer	Project	Key Responsibilities
Domain	Nexora.Domain	Entities, Enums, Value Objects (User, Note, Couple, CalendarEvent, ChatMessage, VaultItem, NewsArticle). Zero third-party dependencies.
Application	Nexora.Application	CQRS Handlers (MediatR), DTOs, Service Interfaces (INexoraDbContext, IAIProvider, ITokenService, ICurrentUserService).
Infrastructure	Nexora.Infrastructure	EF Core DbContext, Fluent Configurations, SQL Server Migrations, SignalR Hub, Hangfire, AI Adapters (OpenAI / Null).
Web API	Nexora.Api	REST Controllers, JWT authentication, Swagger UI, Global Exception Middleware, Production security headers (HSTS, CSP, X-Frame).
Admin Portal	Nexora.Admin	Blazor Server administrative dashboard for metrics, feature toggling, user oversight, and audit trails.
Mobile App	nexora_mobile	Cross-platform Flutter application (iOS/Android/Web) with Provider state management and secure token storage.

Zero-Trust Security & Privacy Model

• **Client Untrust:** Client-supplied identifiers (UserId, CoupleId, AdminStatus) are strictly ignored. Identity is extracted exclusively from cryptographic JWT claims.
• **Privacy Isolation:** Entities support *VisibilityLevel* (Private, Shared, PartnerVisible). Server-side queries strictly filter out partner private items (Notes, Events, Vault Secrets).
• **Admin Segregation:** Administrators can view system audit logs and feature toggles, but database queries structurally forbid admin access to private user messages or journals.

2. Complete Capabilities Matrix (Phases 0–13)

Phase	Delivered Features & Technical Details
Phase 0 — Product Specification	Full specification baseline: SRS.md, ARCHITECTURE.md, SECURITY.md, DATABASE.md, DEVELOPMENT.md, and ADR-001 through ADR-005.
Phase 1 — Engineering Foundation	Monorepo structure initialized with Git branches (main/develop), .NET 10 solution, Flutter mobile scaffold, Blazor admin, and GitHub Actions CI.
Phase 2 — Identity & Security	JWT authentication with refresh tokens, PBKDF2 password hashing, Register/Login/RefreshToken CQRS, and Flutter login/registration screens.
Phase 3 — Platform Core	VisibilityLevel privacy model, FeatureFlag entity with in-memory caching, FeatureFlagService, and AuditLogService for security tracking.

Phase 4 — Admin Foundation	Blazor Server dashboard with live KPI counters, real-time feature flag toggle switches, and security audit log review table.
Phase 5 — Personal Domain Core	Personal Notes (pinned/archived) and Reminders (due dates/recurrence). REST controllers and Flutter Notes/Reminder manager.
Phase 6 — Couple Domain	Couple pairing workflow: 6-character claim codes, invitation acceptance, and server-enforced rule of exactly 1 active couple per user.
Phase 7 — Shared Calendar & Realtime	Calendar events with partner visibility boundaries, SignalR NexoraHub (/hubs/nexora) with couple group broadcasts, and mobile calendar UI.
Phase 8 — Couple Realtime Chat	ChatMessage entity with delivery tracking (Sent/Delivered/Read), chronological history pagination, SignalR push notifications, and bubble chat UI.
Phase 9 — Knowledge Vault	Secure Documents and Secrets manager with tags, client encryption payload support, and strict partner isolation for private secrets.
Phase 10 — AI Platform & Coach	Provider-independent IAIProvider pipeline (OpenAI / Ollama / Null fallback), Ask AI query, and empathetic Communication Coach with tone insights.
Phase 11 — News Platform	Multi-sector news aggregation (Tech, AI, Business, Science, World), canonical URL deduplication, curation ranking, and mobile news feed with Top Story badges.
Phase 12 — Search & Intelligence	Global unified search aggregating across Notes, Reminders, Calendar, Vault, and News with zero partner privacy leakage. Domain-typed UI chips.
Phase 13 — Production Hardening	Multi-stage Dockerfile, docker-compose orchestration (SQL Server + API), HSTS, and HTTP security headers (X-Frame, X-Content-Type, CSP).

3. Published Binaries & Deployment Artifacts

Both backend server applications have been compiled in **Release mode** using .NET 10 and published directly for IIS hosting:

Application	Publish Path	Port / Protocol	Entry Point
Nexora Web API	A:\Projects\Nexora\publish\api	HTTP 5000 / HTTPS 5001	Nexora.Api.dll (ASP.NET Core Module v2)
Nexora Admin	A:\Projects\Nexora\publish\admin	HTTP 5050 / HTTPS 5051	Nexora.Admin.dll (Blazor Server)
Mobile Client	A:\Projects\Nexora\mobile\nexora_mobile	Android / iOS / Web	Flutter App connecting to API baseUrl

4. Step-by-Step IIS Hosting Guide

Step 1: Install Required IIS Components & .NET Hosting Bundle

1. Open Windows **Turn Windows features on or off**.
2. Ensure **Internet Information Services** is checked, including **World Wide Web Services** → **Application Development Features** → **WebSocket Protocol** (critical for SignalR and Blazor Server).
3. Download and install the **ASP.NET Core 10.0 Hosting Bundle** from Microsoft. This installs the native AspNetCoreModuleV2 inside IIS.
4. Restart IIS by running iisreset in an elevated PowerShell terminal.

Step 2: Create Application Pools in IIS Manager

Open **IIS Manager** (inetmgr) and create two dedicated application pools:
• **NexoraApiPool:** .NET CLR Version = No Managed Code, Managed Pipeline Mode = Integrated.
• **NexoraAdminPool:** .NET CLR Version = No Managed Code, Managed Pipeline Mode = Integrated.
Note: Setting 'No Managed Code' is mandatory for ASP.NET Core because IIS acts as a reverse proxy to Kestrel.

Step 3: Create IIS Websites or Virtual Applications

In IIS Manager under **Sites**, add the following websites (or virtual directories under Default Web Site):
1. Nexora Web API:
- Site Name: Nexora.Api
- Application Pool: NexoraApiPool
- Physical Path: A:\Projects\Nexora\publish\api
- Binding: Type = http, Port = 5000 (or IP/hostname of choice)

2. Nexora Admin Portal:
- Site Name: Nexora.Admin
- Application Pool: NexoraAdminPool
- Physical Path: A:\Projects\Nexora\publish\admin
- Binding: Type = http, Port = 5050

Step 4: Configure Folder Permissions

The IIS worker process accounts need read and execute access to the published folders:
Run these commands in an elevated PowerShell terminal:

```
icacls "A:\Projects\Nexora\publish\api" /grant "IIS AppPool\NexoraApiPool":(OI)(CI)RX /T
icacls "A:\Projects\Nexora\publish\admin" /grant "IIS AppPool\NexoraAdminPool":(OI)(CI)RX /T
```

Step 5: Configure SQL Server Database Connection & Secrets

In A:\Projects\Nexora\publish\api\appsettings.json (or via IIS Environment Variables):
1. Verify ConnectionStrings:DefaultConnection points to your SQL Server instance.
2. Provide your secure Jwt:SecretKey (minimum 32 characters).
3. On first startup, the API will automatically execute pending EF Core migrations and seed initial roles and feature flags.

5. User Guide: How to Use and Test the Platform

A. Testing the Web API

1. Open your browser and navigate to http://localhost:5000/health. You should receive an Healthy status confirming database connectivity.
2. To explore endpoints, navigate to http://localhost:5000/swagger (when in Development) or use Postman to test:
   - POST /api/auth/register — Register a user.
   - POST /api/auth/login — Obtain JWT Bearer token.
   - GET /api/notes, GET /api/calendar, GET /api/vault, GET /api/news, GET /api/search?q=...

B. Accessing the Blazor Admin Portal

1. Navigate to http://localhost:5050 in your browser.
2. View the system dashboard showing total users, active couples, system health, and Hangfire job status.
3. Use the **Feature Flags** panel to toggle flags on/off dynamically with immediate server-side enforcement.

C. Running the Flutter Mobile App

From the workspace terminal, launch the mobile client against your local IIS backend:
cd mobile/nexora_mobile
flutter run -d chrome (or android emulator)

Core Mobile Workflows to Experience:
- **Personal Mode:** Create personal notes and set due-date reminders.
- **Couple Pairing:** Generate an invite code from Partner 1, switch to Partner 2, and enter the code to pair.
- **Realtime Chat & Calendar:** Exchange SignalR messages with instant push updates and schedule shared events.
- **Knowledge Vault:** Store private passwords or shared lease documents with granular privacy tags.
- **AI Communication Coach:** Type an emotional message and receive empathetic phrasing suggestions.
- **Curated News & Global Search:** Browse multi-sector articles and use unified search across all domains.

What's Next?
- **SSL Certificate:** Bind an HTTPS certificate (e.g. via Win-ACME / Let's Encrypt) to your IIS sites for production encryption.
- **CI/CD Deployment:** Connect the GitHub Actions workflow in .github/workflows/ci.yml to automatically deploy published artifacts upon merge to main.
- **Mobile Store Release:** Run flutter build apk --release or flutter build appbundle for Google Play and TestFlight distribution.